EAD Unifor

PROGRAMAÇÃO FUNCIONAL

4 Programação funcional no desenvolvimento de aplicações





Este trabalho está licenciado com uma Licença Creative Commons Atribuição-NãoComercial-SemDerivações 4.0 Internacional.

APLICAÇÕES DA PROGRAMAÇÃO FUNCIONAL NO DESENVOLVIMENTO DE APLICAÇÕES

Sumário

1. Vinculação com banco de dados, validação de acesso e restrições, cadastro, consulta, atualização e remoção de itens

2. Recursos de framework: scaffolds e geração de boilerplates e os diferenciais tecnológicos

3. Suporte à Server Side Rendering, expressividade, abstração, ausência de variáveis e estado

4. Perspectivas de recuperação de falha e validação da aplicação e teste de stress



Sumário clicável

Olá



A programação funcional é um paradigma que prioriza funções como base para resolver problemas e criar programas. Sua popularidade no desenvolvimento de aplicações tem crescido devido a várias vantagens, como redação de código conciso e legível, progressão ágil no desenvolvimento e facilidade de manutenção futura. Oferece recursos poderosos, como funções de alta ordem, composição funcional, imutabilidade de dados e recursão, permitindo resolver problemas complexos de forma elegante e eficiente. Na área de desenvolvimento de aplicações web, frameworks, como React e Elm, facilitam a criação de interfaces interativas e responsivas, além de simplificar o gerenciamento do estado da aplicação. Bibliotecas como Apache Spark, em Scala, possibilitam a execução de cálculos distribuídos e o processamento eficiente de grandes volumes de dados. A imutabilidade dos dados é especialmente relevante para garantir a integridade de informações sensíveis.

1. Vinculação com banco de dados, validação de acesso e restrições, cadastro, consulta, atualização e remoção de itens

Quando desenvolvemos aplicativos ou sistemas que precisam armazenar dados persistentes, o uso de um banco de dados é essencial. No paradigma funcional, existem várias bibliotecas disponíveis para interagir com bancos de dados, permitindo que você crie, leia, atualize e exclua dados (CRUD) de forma eficiente.

O CRUD é um acrônimo que representa as quatro principais operações de banco de dados: Create (Criação), Read (Leitura), Update (Atualização) e Delete (Exclusão). Essas operações formam a base para a manipulação de dados em um banco de dados e são amplamente utilizadas em aplicações do mundo real.

Abaixo, vamos apresentar um exemplo de como se conectar ao banco de dados Universidade no MongoDB utilizando a linguagem Haskell.

```
{-# LANGUAGE OverloadedStrings #-}
import Database.MongoDB

main :: IO ()
main = do

//Estabeleça uma conexão com o banco de dados MongoDB
pipe <- connect (host “localhost”)

/Selecione o banco de dados
let database = “Universidade”
```

A criação (Create) envolve a adição de novos registros ou objetos ao banco de dados. Por exemplo, ao criar um aplicativo de gerenciamento de tarefas, você pode criar uma nova tarefa e inseri-la no banco de dados.

```
//Informamos ao banco que vamos trabalhar com a coleção EstudantesB
let collection = "Estudantes"

//Insira alguns documentos na coleção "Estudantes"
insertDocuments pipe database collection

//Feche a conexão com o banco de dados
close pipe

//Função para inserir documentos na coleção "Estudantes"
insertDocuments :: Pipe -> Database -> Collection -> IO ()
insertDocuments pipe database collection = do
```

```
//Crie um documento representando um estudante
let aluno01 = ["nome" =: "João", "idade" =: 20, "curso" =: "Ciência
da Computação"]
let aluno02 = ["nome" =: "Maria", "idade" =: 22, "curso" =:
"Engenharia  da Computação"]

//Insira os documentos na coleção
_ <- access pipe master database $ insertMany collection [aluno01,
aluno02]

putStrLn "Alunos  inseridos com sucesso!"
```

A leitura (Read) refere-se à recuperação de dados existentes do banco de dados. Por exemplo, em um aplicativo de blog, você pode ler os artigos de um autor específico ou exibir uma lista de todos os usuários registrados.

```
//Selecione a coleção
let collection = "Estudantes"

//Recupere os estudantes com atributo da "Engenharia da Computação"
retrieveStudents pipe database collection

//Feche a conexão com o banco de dados
close pipe

//Função para recuperar estudantes da "Engenharia da Computação"
retrieveStudents :: Pipe -> Database -> Collection -> IO ()
retrieveStudents pipe database collection = do

//Crie uma consulta para recuperar os estudantes
let query = select ["curso" =: "Engenharia da Computação"]
collection

//Execute a consulta e recupere os documentos correspondentes
documents <- access pipe master database $ find query >>= rest

//Exiba os documentos recuperados
putStrLn "Estudantes com curso Engenharia da Computação:"
mapM_ print documents
```

A atualização (Update) envolve a modificação de dados existentes no banco de dados. Por exemplo, em um aplicativo de comércio eletrônico, você pode atualizar as informações de um produto, como o preço ou a descrição. Conforme podemos observar, o código abaixo vai atualizar o curso de todos os alunos de Engenharia da Computação para Análise e Desenvolvimento de Sistemas:

```
//Atualizar curso "Engenharia da Computação" para "ADS"
let updateQuery = select ["curso" =: "Engenharia da Computação"]
collection
let updateFields = ["$set" =: ["curso" =: "ADS"]]
_ <- access pipe master database $ modify updateQuery updateFields

//Recupere novamente os estudantes para verificar a atualização
updatedStudents <- access pipe master database $ find updateQuery >>=
rest

//Exiba os estudantes atualizado
putStrLn "Estudantes atualizados:"
mapM_ print updatedStudents
```

Usamos a função modify para executar a atualização, passando a consulta updateQuery e os campos de atualização updateFields. Após a atualização, recuperamos novamente os estudantes usando a mesma consulta updateQuery para verificar a atualização. Os estudantes atualizados são armazenados na variável updatedStudents.

A exclusão (Delete) é a remoção de registros ou objetos do banco de dados. Por exemplo, em um aplicativo de agenda, você pode excluir um evento que não é mais relevante. Para exemplificar a exclusão, vamos deletar de nosso Banco de Dados todos os estudantes que cursam Informática.

```
//Exclua os estudantes de Informática
let deleteQuery = select ["curso" =: "Informática"] collection
_ <- access pipe master database $ delete deleteQuery

//Recupere novamente os estudantes para verificar a exclusão
remainingStudents <- access pipe master database $ find (select []
collection) >>= rest

//Exiba os estudantes restantes
putStrLn "Estudantes restantes:"
mapM_ print remainingStudents
```

Como podemos observar no exemplo, criamos uma nova consulta chamada deleteQuery para selecionar os estudantes com o atributo “curso” igual a “Informática”. Em seguida, usamos a função delete para excluir os estudantes correspondentes à consulta deleteQuery. Após a exclusão, recuperamos novamente os estudantes usando a consulta select [ ] para selecionar todos os estudantes restantes na coleção. Por fim, exibimos os estudantes restantes usando a função print.

Agora que já sabemos implementar um CRUD em Haskell, vamos melhorar a segurança da nossa aplicação através da aplicação da validação de acesso que se refere à aplicação de regras que controlam quais usuários têm permissão para executar operações em determinadas coleções ou bancos de dados. Essas regras podem ser baseadas em critérios como papéis de usuário, permissões específicas ou qualquer lógica personalizada definida pelo desenvolvedor.

Voltar ao sumário

A biblioteca mongoDB em Haskell permite definir validações de acesso usando o comando createCollection. Aqui está um exemplo de código que cria uma coleção “Estudantes” e define uma validação de acesso para permitir apenas operações de leitura (READ) na coleção:

```
import Database.MongoDB

main :: IO ()
main = do
  //Estabeleça uma conexão com o banco de dados MongoDB
  pipe <- connect (host "localhost")

  //Selecione o banco de dados
  let database = "Universidade"

  //Selecione a coleção
  let collection = "Estudantes"

  //Defina a validação de acesso para permitir apenas operações de
leitura
  let accessValidation = ["$jsonSchema" =: ["bsonType" =: "object",
"readOnly" =: True]]
  _<- access pipe master database $ createCollection collection
[("validator", accessValidation)]

  //Feche a conexão com o banco de dados
  close pipe
```

Neste exemplo, usamos a função createCollection com o parâmetro “validator” para definir uma validação de acesso na coleção “Estudantes”. A validação de acesso é definida como um documento JSON Schema que especifica que os documentos na coleção são somente leitura (“readOnly” =: True).

As restrições em um banco de dados são regras definidas para garantir a integridade dos dados, limitando os valores permitidos em campos específicos das tabelas ou coleções. Elas são aplicadas para evitar que dados inválidos sejam inseridos ou atualizados no banco de dados, mantendo a consistência dos dados e garantindo a conformidade com as regras de negócio.

Quadro 1 – Restrições de bancos de dados

Integridade dos dados	Garante que apenas dados válidos sejam armazenados no banco de dados, protegendo-os contra valores inválidos ou inconsistentes. Isso ajuda a manter a qualidade e a confiabilidade dos dados, evitando a corrupção ou a inserção de informações que não estejam de acordo com as regras do sistema.
Consistência dos dados	Ao impor restrições, é possível garantir que as informações armazenadas estejam consistentes em todo o banco de dados. Isso evita a ocorrência de dados duplicados ou conflitantes, mantendo a integridade e a coerência dos dados em diferentes partes da aplicação.
Aplicação de regras de negócio	Permite que você aplique regras de negócio diretamente no nível do banco de dados. Isso ajuda a garantir que as regras e restrições definidas pela organização sejam cumpridas, mantendo a conformidade e a consistência dos dados em conformidade com as políticas da empresa.

Fonte: EAD Unifor (2023)

Abaixo vamos apresentar um exemplo codificado em Haskell sobre o uso de restrições em uma coleção Produtos para garantir que exista uma restrição sobre o atributo Preço, a fim de garantir que o preço sempre tenha valores maiores do que zero.

```
import Database.MongoDB

main :: IO ()
main = do
  pipe <- connect (host "localhost")

  //Selecione o banco de dados
  let database = "Loja"

  //Selecione a coleção
  let collection = "Produtos"

  //Adicione uma restrição para garantir que o campo "preco" seja
maior que zero
  let priceConstraint = ["preco" =: 1]
    _ <- access pipe master database $ createIndex (Index collection
priceConstraint ["unique" =: True, "partialFilterExpression" =:
["preco" =: ["$gt" =: 0]]])

//Feche a conexão com o banco de dados
close pipe
```

Porém, as restrições apresentam desafios, como:

Quadro 2 – Desafios das restrições

Complexidade	A definição e o gerenciamento de restrições podem adicionar complexidade ao desenvolvimento e à manutenção do banco de dados. Restrições mal projetadas ou mal implementadas podem dificultar a inserção, atualização ou exclusão de dados válidos.
Desempenho	Restrições complexas podem impactar o desempenho do banco de dados, especialmente durante operações que envolvem grandes volumes de dados. A verificação das restrições pode exigir recursos adicionais e aumentar o tempo de processamento.
Restrições rígidas	Restrições muito rígidas podem restringir excessivamente as operações de modificação de dados, causando dificuldades em casos legítimos. Isso pode levar a problemas de escalabilidade e flexibilidade na manipulação dos dados.

Fonte: EAD Unifor (2023)

É importante encontrar um equilíbrio adequado ao definir restrições no banco de dados, levando em consideração as necessidades específicas do sistema, as regras de negócio e o desempenho desejado, desempenhando um papel fundamental na manutenção da integridade e da qualidade dos dados em um banco de dados.

Embora apresentem alguns desafios, os benefícios superam em muito as limitações apresentadas, tornando sua adoção uma prática recomendada para garantir dados consistentes e confiáveis em um sistema de armazenamento de dados.

Também, aprendemos sobre a validação de acesso, que desempenha um papel fundamental, em que a imutabilidade dos dados é valorizada e a pureza das funções é enfatizada, a validação de acesso garante que as operações sejam realizadas somente por usuários autorizados e dentro das restrições definidas, garantindo: Segurança, Controle de Acesso Granular, Conformidade com Regulamentações, Integridade e Coerência dos Dados, Auditoria e Rastreabilidade.

2. Recursos de framework: scaffolds e geração de boilerplates e os diferenciais tecnológicos

Vamos agora estudar sobre recursos de framework. Esses, geralmente, fornecem conjuntos de bibliotecas, ferramentas e padrões que facilitam o desenvolvimento de aplicativos seguindo os princípios da programação funcional.

Figura 1 - Principais recursos explorados pelos frameworks



Fonte: EAD Unifor (2023)

Esses são apenas alguns dos recursos comuns encontrados nos frameworks de linguagens funcionais. É importante observar que a disponibilidade e a implementação desses recursos podem variar entre as diferentes linguagens e frameworks funcionais.

O conceito de scaffolds se refere às ferramentas que auxiliam na criação rápida e automatizada de estruturas básicas de um projeto, como diretórios, arquivos de código-fonte e configurações iniciais. Essas ferramentas são projetadas para acelerar o processo

de inicialização de um novo projeto, fornecendo uma estrutura básica que pode ser personalizada posteriormente.

Quadro 3 – Vantagens do uso de scaffolds

Produtividade	Com um scaffold, é possível iniciar rapidamente um novo projeto, evitando a configuração manual de diretórios e arquivos básicos. Isso economiza tempo e permite que o desenvolvedor se concentre na implementação das funcionalidades do aplicativo.
Padronização	Os scaffolds seguem convenções e boas práticas estabelecidas pela comunidade, facilitando a criação de projetos consistentes. Essa padronização é especialmente útil quando vários desenvolvedores trabalham no mesmo projeto, pois todos seguem a mesma estrutura, tornando o código mais fácil de entender e manter.
Customização	Apesar de fornecerem uma estrutura inicial, os scaffolds permitem personalizar e ajustar o projeto de acordo com as necessidades específicas do desenvolvedor. É possível adicionar ou remover componentes, configurar dependências e fazer ajustes para atender aos requisitos do projeto. Essa flexibilidade permite que o desenvolvedor tenha controle sobre o código e a arquitetura do aplicativo.

Fonte: EAD Unifor (2023)

Para implementar nosso exemplo, como usar o scaffold “stack” em Haskell, precisamos inicialmente instalar o Stack, que é um gerenciador de pacotes e uma ferramenta de build para Haskell, e, posteriormente, dentro do terminal, navegar para o diretório onde desejamos criar o projeto e executar o seguinte comando:

```
stack new meuProjetoUnifor
```

Isso criará uma nova pasta chamada **meuProjetoUnifor** com uma estrutura básica para o projeto Haskell, incluindo arquivos como “meuProjetoUnifor.cabal” (arquivo de configuração do projeto) e uma pasta “src” (onde você pode colocar seu código-fonte).

Você pode personalizar o projeto ajustando o arquivo “meuProjetoUnifor.cabal” para adicionar dependências ou configurar outros aspectos do projeto. Em seguida, você pode começar a implementar seu código-fonte nos arquivos dentro da pasta “src”.

O exemplo acima demonstra o uso básico do scaffold “stack” em Haskell. Existem outros scaffolds disponíveis, como “cabal”, que também fornecem estruturas iniciais para projetos Haskell.

Outro exemplo pode ser observado abaixo:

```haskell
isPrimo :: Int -> Bool
isPrimo n
  | n <= 1 = False
  | otherwise = all (\x -> n `mod` x /= 0) [2..isqrt n]
  where
    isqrt = floor . sqrt . fromIntegral

main :: IO ()
main = do
  putStrLn "Verificação de número primo"
  putStrLn "Digite um número inteiro: "

  input <- getLine
  let numero = read input :: Int
  let resultado = isPrimo numero
  putStrLn (mensagemResultado resultado)

mensagemResultado :: Bool -> String
mensagemResultado True = "É um número primo."
mensagemResultado False = "Não é um número primo."
```

No exemplo acima, o scaffolding é implementado por meio da definição da função **mensagemResultado**. Essa função recebe um valor booleano indicando se o número é primo ou não e retorna uma mensagem correspondente. Ela fornece um suporte adicional ao formatar e exibir o resultado da verificação.

Quadro 4 – Desafios do uso de scaffolds

Curva de aprendizado	Alguns scaffolds podem apresentar uma curva de aprendizado inicial, especialmente para aqueles que não estão familiarizados com a ferramenta específica. Isso pode exigir algum tempo para compreender as convenções e personalizações possíveis oferecidas pelo scaffold. Entretanto, uma vez que o desenvolvedor adquire conhecimento, a produtividade pode aumentar significativamente.
Rigidez	Alguns scaffolds podem ser rígidos, em termos de estrutura e convenções, o que pode limitar a flexibilidade do desenvolvedor, caso seja necessário desviar dos padrões estabelecidos pelo scaffold. Essa rigidez pode ser uma desvantagem em cenários onde os requisitos específicos do projeto não se alinham perfeitamente com a estrutura imposta pelo scaffold, exigindo mais esforço para adaptar o projeto conforme necessário.

Fonte: EAD Unifor (2023)

Dentro desse contexto, o uso de boilerplate na programação funcional refere-se a trechos de código repetitivos e padronizados que são necessários para realizar tarefas comuns, mas que não contribuem diretamente para a lógica ou a funcionalidade principal do programa. Esses trechos de código podem incluir importações de bibliotecas, definições de tipos de dados, configurações de ambiente e outras estruturas básicas.

Simon Peyton Jones define o boilerplate como "**um código que não está diretamente relacionado à tarefa principal do programa, mas ainda assim é necessário para que o programa funcione corretamente**."

A partir deste conceito, vamos apresentar um exemplo de boilerplate implementado na linguagem Haskell, considerando um programa simples que lê um número inteiro e imprime o seu dobro:

```
import Text.Read (readMaybe)
main :: IO ()
main = do
  putStrLn "Digite um número inteiro:"
  input <- getLine
  case readMaybe input of
    Just number -> do
      let doubled = number * 2
      putStrLn $ "O dobro do número é: " ++ show doubled
    Nothing -> putStrLn "Entrada inválida. Por favor, digite um
número inteiro."
```

No exemplo acima, o trecho de código responsável pela importação da função readMaybe da biblioteca Text.Read é considerado **boilerplate**, pois é necessário para realizar a conversão segura da entrada do usuário para um número inteiro. No entanto, ele não está diretamente relacionado à lógica principal do programa, que é calcular o dobro do número.

Assim, o **boilerplate** é uma parte inevitável da programação funcional, mas pode ser minimizado ou abstraído para tornar o código mais conciso e legível, focando na lógica específica do problema em questão.

```
module Main where

isPrimo :: Int -> Bool
isPrimo n
  | n <= 1 = False
  | otherwise = all (\x -> n `mod` x /= 0) [2..isqrt n]
  where
    isqrt = floor . sqrt . fromIntegral

main :: IO ()
main = do
  putStrLn "Verificação de número primo"
  putStrLn "Digite um número inteiro: "

  input <- getLine
  let numero = read input :: Int

  let resultado = isPrimo numero
  putStrLn (mensagemResultado resultado)
```

```
mensagemResultado :: Bool -> String
mensagemResultado True = "É um número primo."
mensagemResultado False = "Não é um número primo."
```

Neste segundo exemplo, o código foi organizado como um boilerplate básico em Haskell. O module Main where indica que este é o módulo principal. A função isPrimo e a mensagemResultado foram definidas logo abaixo, seguidas pela função main.

O main contém a lógica principal do programa, onde ocorre a interação com o usuário. O código solicita ao usuário que digite um número, converte-o para o tipo Int e, em seguida, chama a função isPrimo para verificar se o número é primo. Por fim, exibe a mensagem correspondente usando a função mensagemResultado. Por fim, vamos apresentar um exemplo que combina a utilização do scaffold com boilerplate:

```
module Main where

data Person = Person
  { name :: String
  , age :: Int
  }

createPerson :: String -> Int -> Person
createPerson n a = Person n a

printPerson :: Person -> IO ()
printPerson p = do
  putStrLn "Dados da pessoa:"
  putStrLn ("Nome: " ++ name p)
  putStrLn ("Idade: " ++ show (age p))

main :: IO ()
main = do

  putStrLn "Cadastro de Pessoa"
  putStrLn "Digite o nome da pessoa: "
  nome <- getLine
  putStrLn "Digite a idade da pessoa: "
  idadeInput <- getLine

  let idade = read idadeInput :: Int
  let pessoa = createPerson nome idade

  putStrLn "\nDados cadastrados:"
  printPerson pessoa
```

No código acima, temos uma estrutura de dados Person que representa uma pessoa com um nome e uma idade. A função createPerson é responsável por criar uma nova instância de Person com base nos parâmetros fornecidos. O código também possui a

função printPerson que recebe uma pessoa como argumento e imprime seus dados na saída.

No main, o programa solicita ao usuário o nome e a idade de uma pessoa. Em seguida, chama a função createPerson para criar uma instância de Person com esses dados. Por fim, exibe os dados cadastrados chamando a função printPerson. O código fornece tanto um scaffolding quanto um boilerplate.

O scaffolding é representado pelas funções createPerson e printPerson, que fornecem a estrutura e as funcionalidades básicas para criar e imprimir informações sobre uma pessoa. O boilerplate está presente na estrutura do módulo, definição de tipos de dados e funções principais (main). Essa combinação de scaffolding e boilerplate facilita a criação e a organização do código, fornecendo a estrutura inicial necessária e as funcionalidades básicas para a implementação de um programa mais completo.

Para concluir nosso Circuito, vale lembrar que o scaffolding se refere à estrutura de suporte que é gradualmente removida à medida que o desenvolvedor ganha mais habilidades e conhecimento. Ele fornece um ponto de partida claro, orientação e suporte durante o processo de desenvolvimento. Por outro lado, o boilerplate se refere ao código padrão necessário para iniciar um projeto ou implementar determinadas funcionalidades básicas. É a parte repetitiva e necessária do código que precisa ser escrita ou configurada em várias situações semelhantes. O boilerplate geralmente inclui definição de tipos, estrutura de módulos, importações e funções principais.

Ao combinar o scaffolding e o boilerplate, é possível fornecer uma estrutura inicial sólida, funcionalidades básicas e exemplos concretos para ajudar no desenvolvimento de software. O scaffolding ajuda os desenvolvedores a entenderem e aplicarem os conceitos de forma progressiva, enquanto o boilerplate oferece a estrutura e as funcionalidades essenciais para o funcionamento do programa.

Ao utilizar o scaffolding e o boilerplate de forma eficaz, é possível acelerar o processo de desenvolvimento, promover a compreensão conceitual e aprimorar a produtividade, permitindo que os alunos ou desenvolvedores se concentrem em partes mais complexas e específicas do código.

Compreendidos os assuntos tratados até aqui? Então, vamos aprender mais, no nosso próximo Circuito de estudo!

3. Suporte a Server Side Rendering, expressividade, abstração, ausência de variáveis e estado

A programação funcional tem ganhado cada vez mais popularidade devido às suas vantagens em termos de legibilidade, expressividade e facilidade de manutenção. Uma das abordagens poderosas na programação funcional é o conceito de Server Side Rendering (SSR), que permite renderizar páginas no lado do servidor antes de enviá-las ao cliente. Neste contexto, Haskell, uma linguagem de programação funcional pura, destaca-se como uma opção sólida para implementar SSR em aplicações web.

A seguir, apresentaremos um exemplo no qual exploraremos como criar um servidor web de Server Side Rendering (SSR) em Haskell usando o framework Yesod. Yesod é uma biblioteca web robusta e extensível que oferece suporte para a construção de aplicações web seguras e eficientes. Através deste exemplo simples, veremos como renderizar uma página HTML básica contendo informações fornecidas pelo servidor.

Usando o Yesod, definiremos o roteamento para a página inicial e criaremos uma função para a lógica de renderização. Com isso, conseguiremos gerar uma página completa e funcional no servidor, enviando-a pronta para o cliente, o que traz benefícios como melhoria de desempenho e otimização para mecanismos de busca.

Ao final, teremos uma introdução prática ao conceito de Server Side Rendering em Haskell com Yesod, mostrando como essa combinação pode ser poderosa para desenvolver aplicações web robustas e eficientes em uma linguagem funcional de alto desempenho.

Para implementar o exemplo, inicialmente vamos criar um novo projeto Yesod. Para isso, vamos executar os seguintes comandos:

```
stack new ssr-example yesodweb/simple
cd ssr-example
stack setup
```

A seguir, vamos realizar a definição do roteamento e a lógica para renderizar a página. Para isso, vamos criar um roteamento no arquivo config/routes:

```
GET HomeR HomeR GET
```

Agora, definimos a lógica para renderizar a página no arquivo Handler/Home.hs:

```
module Handler.Home where

import Import

getHomeR :: Handler Html
getHomeR = defaultLayout $ do
    setTitle "Server Side Rendering in Haskell"
    [whamlet|
        <h1> Alô mundo, Server Side Rendering implementado em
Haskell!</h1>
        <p> Essa página foi renderizada em Yesod.
    |]
```

No terminal, execute o comando abaixo para iniciar o servidor:

```
stack run
```

Depois acesse a página no endereço: http://localhost:3000/home. A partir do seu navegador, para visualizar a página renderizada com o texto:

```
Alô mundo, Server Side Rendering implementado em  Haskell!
Essa página foi renderizada em Yesod.
```

Neste exemplo, utilizamos o Yesod para definir um roteamento para a página inicial (Home) e, em seguida, criamos uma função getHomeR que renderiza o conteúdo da página. A função defaultLayout é usada para renderizar o layout padrão do Yesod, que inclui o cabeçalho da página com o título definido, seguido do conteúdo específico da página.

O SSR ocorre porque a página é renderizada no servidor e enviada ao cliente já com o conteúdo preenchido. Isso é útil para melhorar o desempenho e a indexação de mecanismos de busca, pois o conteúdo da página está disponível desde o início, sem a necessidade de esperar pelo carregamento do JavaScript para renderizar o conteúdo no lado do cliente.

Existem algumas otimizações que podem ser aplicadas para melhorar o desempenho do Server Side Rendering (SSR). Abaixo vamos apresentar algumas que consideramos importantes:

Figura 2 – Otimizações de desempenho de SSR



Fonte: EAD Unifor (2023)

Na programação funcional, o conceito de expressividade refere-se à capacidade da linguagem em permitir que os programadores expressem suas ideias e lógica de forma clara, concisa e legível. Uma linguagem expressiva como Haskell permite que os desenvolvedores expressem soluções de maneira próxima à linguagem natural, facilitando a compreensão e a manutenção do código.

Quadro 4 – Características da expressividade em Haskell

Notação de Funções	Permite definir funções de alta ordem e funções anônimas (lambda), simplificando a abstração e a composição de comportamentos, tornando o código mais claro e conciso.
Tipagem Estática e Inferência de Tipos	A tipagem estática identifica erros em tempo de compilação, garantindo operações em tipos compatíveis. A inferência de tipos permite ao compilador deduzir tipos de expressões, reduzindo declarações verbosas de tipos.
Pattern Matching	Permite combinar e desestruturar dados de forma concisa, facilitando a manipulação de estruturas complexas.
Recursão e Compreensão de Listas	Suporta a recursão de maneira natural, permitindo a definição elegante e concisa de algoritmos. A compreensão de listas simplifica a manipulação de listas.

Fonte: EAD Unifor (2023)

Agora, vamos apresentar um exemplo simples para demonstrar a expressividade em Haskell. Neste exemplo, vamos definir uma função para calcular o fatorial de um número.

```
//Definição da função fatorial em Haskell
fatorial :: Integer -> Integer
fatorial 0 = 1
fatorial n = n * fatorial (n - 1)
```

Neste exemplo, a função fatorial é definida usando pattern matching. Ela recebe um inteiro n e calcula o fatorial de n através da recursão. A definição é objetiva, aproveitando o pattern matching e a recursão de forma expressiva. Agora, podemos chamar a função fatorial para calcular o fatorial de um número específico:

```
main :: IO ()
main = do
    let numero = 5
    putStrLn $ "O fatorial de " ++ show numero ++ " é " ++ show
(fatorial numero)
```

A saída será:

```
"O fatorial de 5 é 120".
```

De forma a terminar os conceitos definidos neste Circuito e tornar seu entendimento mais simples, vamos apresentar um exemplo em Haskell para abordar os conceitos de abstração, ausência de variáveis e estado. Neste exemplo, vamos implementar uma função para calcular o quadrado de uma lista de números sem usar variáveis mutáveis ou armazenar estados intermediários.

Faremos uso da **abstração** ao criar uma função que encapsula a lógica para calcular

o quadrado dos elementos de uma lista. Isso nos permitirá reutilizar essa função em diferentes contextos sem se preocupar com detalhes de implementação.

Na programação funcional, evitamos o uso de variáveis mutáveis. Em vez disso, usamos funções para transformar dados e produzir novos valores. No exemplo, não usaremos variáveis para armazenar os resultados intermediários.

A ausência de estado significa que não haverá efeitos colaterais ou alteração do estado do programa. Em nosso exemplo, a função calculará o quadrado dos elementos, mas não haverá alterações no estado global do programa.

```
//Definição da função para calcular o quadrado dos elementos da lista
quadradoLista :: [Int] -> [Int]
quadradoLista = map (\x -> x * x)
```

Neste exemplo, a função quadradoLista é uma função de alta ordem que usa a função map. Ela recebe uma lista de inteiros e, em seguida, aplica a função \x -> x * x a cada elemento da lista. Essa função lambda é responsável por calcular o quadrado de cada elemento da lista.

A função map é uma função de ordem superior que aplica uma função a cada elemento de uma lista e retorna uma nova lista com os resultados das aplicações. Ela encapsula a lógica de percorrer a lista e aplicar a função a cada elemento, o que torna o código mais abstrato e reutilizável.

Por fim, vamos usar essa função para calcular o quadrado de uma lista de números:

```
main :: IO ()
main = do
    let lista = [1, 2, 3, 4, 5]
    let quadrados = quadradoLista lista
    putStrLn $ "Lista original: " ++ show lista
    putStrLn $ "Quadrados: " ++ show quadrados
```

Ao executar o código acima, obteremos a seguinte saída:

```
Lista original: [1,2,3,4,5]
Quadrados: [1,4,9,16,25]
```

Neste exemplo, mostramos como a programação funcional em Haskell permite a abstração de operações, a ausência de variáveis mutáveis e a falta de efeitos colaterais, tornando o código mais claro e seguro.

Voltar ao sumário

Para concluir nosso passeio pelos temas deste Circuito, vamos lembrar que a adoção do Suporte a Server-Side Rendering (SSR) em aplicações web funcionais tem demonstrado ser uma abordagem valiosa para aprimorar a expressividade, abstração e reduzir a dependência de variáveis e estados durante o desenvolvimento de aplicações. Ao optar pelo SSR, as vantagens são amplas, pois a renderização do lado do servidor possibilita o pré-renderização do conteúdo e sua entrega direta ao cliente. Consequentemente, proporciona uma experiência de usuário mais ágil e eficiente, evitando a espera por solicitações adicionais de recursos.

A **adoção da expressividade** é notadamente aprimorada no emprego de SSR, permitindo aos desenvolvedores criar interfaces mais dinâmicas e interativas. Essa característica torna-se especialmente relevante quando combinada com modernos frameworks do lado do cliente, possibilitando a interação com o conteúdo pré-renderizado e proporcionando uma experiência mais fluida e responsiva aos usuários. Ademais, o SSR promove a abstração do estado da aplicação, o que simplifica a lógica do código e torna a manutenção e a escalabilidade do projeto mais viáveis. Com o estado sendo gerenciado no servidor, os comuns problemas de sincronização de estado entre o cliente e o servidor são mitigados, reduzindo a ocorrência de erros e comportamentos inesperados.

A **ausência de variáveis** e estado no cliente, fortemente adotada pelo paradigma funcional, também pode elevar a segurança, já que dados sensíveis ou críticos podem ser manipulados e armazenados de forma mais segura no servidor, minimizando a exposição potencial a ataques maliciosos. Entretanto, é relevante lembrar que o uso de SSR não se mostra a melhor escolha em todos os casos. Cada aplicação possui suas próprias necessidades e requisitos específicos e, portanto, o desenvolvedor deve avaliar criteriosamente os prós e contras antes de optar por essa abordagem. Em certos cenários, como em aplicações altamente interativas, o uso de SSR pode não ser a solução mais adequada, e a adoção de técnicas, como Client-Side Rendering (CSR) ou Server-Side Rendering Híbrido, pode ser mais indicada.

Tudo ok com os assuntos tratados até aqui? Vamos então para o próximo Circuito de estudos!

4. Perspectivas de recuperação de falha e validação da aplicação e teste de stress

Em um mundo cada vez mais interconectado, onde pessoas são substituídas por aplicações autônomas, inteligentes e que operam sem pausas, a robustez e a confiabilidade dessas aplicações são essenciais e não podem deixar margem para dúvidas.

Em cenários nos quais tais aplicações estão envolvidas em atividades críticas, como monitoramento das condições de pacientes em sistemas médicos ou controle de parâmetros de segurança em um reator nuclear, além de outras aplicações em tempo real, e-commerce e transferência de fundos em sistemas financeiros, a recuperação de falhas torna-se uma funcionalidade indispensável para as aplicações modernas.

O ponto central da recuperação de falhas reside na ideia de que, mesmo diante de erros ou falhas durante a execução do programa, o sistema deve ser capaz de se recuperar ou se reestabelecer a um estado funcional, evitando falhas catastróficas e minimizando o impacto negativo sobre o usuário final.

Nesse contexto, é crucial que as aplicações sejam projetadas para lidar com falhas de maneira proativa, garantindo que os erros sejam devidamente tratados, prevenindo a propagação de problemas e mantendo a continuidade dos serviços. Dessa forma, a recuperação de falhas assume papel fundamental na manutenção da disponibilidade dos sistemas, na preservação da confiança dos usuários e na minimização de impactos financeiros adversos.

À medida que o mundo digital avança, a capacidade de recuperação de falhas se torna um diferencial competitivo para as empresas, que buscam proporcionar experiências cada vez mais confiáveis e satisfatórias para seus clientes. Portanto, investir em estratégias e técnicas de recuperação de falhas é um aspecto crucial do desenvolvimento de aplicações modernas, permitindo que elas se adaptem e persistam diante de um ambiente dinâmico e propenso a falhas, proporcionando assim uma experiência de usuário segura e consistente.

Existem vários mecanismos de recuperação de falhas que podem ser empregados. Vejamos alguns dos desses principais mecanismos a seguir.

Tratamento de exceções é uma técnica comum em muitas linguagens de programação, onde blocos de código são usados para capturar e tratar erros específicos que podem ocorrer durante a execução do programa. Isso permite que o programa tome ações apropriadas quando uma falha é detectada, evitando que o sistema pare de funcionar abruptamente.

Em Haskell, o tratamento de exceções é feito usando a funcionalidade de tipos monádicos, em particular o tipo Either ou o tipo Maybe. Abaixo apresentaremos um exemplo simples

usando o tipo Either, que é frequentemente usado para tratar exceções e erros. Vamos criar uma função que divide dois números, se o segundo número for zero, queremos lançar uma exceção ou retornar um valor indicando o erro.

```
//Definição da função de divisão de dois números com tratamento de exceções
DivisaoSegura:: Double -> Double -> Either String Double
DivisaoSegura _ 0 = Left "Divisão por zero não é permitida."
DivisaoSegura x y = Right (x / y)
```

Neste exemplo, usamos o tipo Either String Double para representar o resultado da divisão. O construtor Left é usado para representar o caso de erro, onde a string contém uma mensagem de erro descrevendo o problema. O construtor Right é usado para representar o caso de sucesso, onde o resultado da divisão é armazenado.

Agora, podemos usar essa função em nosso código e tratar as exceções adequadamente:

```
main :: IO ()
main = do
    putStrLn "Digite o numerador:"
    inputNumerador <- getLine
    putStrLn "Digite o denominador:"
    inputDenominador <- getLine

    let numerador = read inputNumerador :: Double
        denominador = read inputDenominador :: Double

    case DivisaoSegura numerador denominador of
        Left errorMsg -> putStrLn $ "Erro: " ++ errorMsg
        Right result -> putStrLn $ "Resultado da divisão: " ++ show
result
```

Um segundo mecanismo de recuperação de falhas que podemos explorar são as **Retentativas (Retry)**, que consistem em tentar executar novamente uma operação que falhou em determinado momento. Essa estratégia pode ser útil quando a falha é temporária ou causada por problemas transitórios, como problemas de rede.

Podemos implementar o mecanismo de retentativas usando recursão e monads, principalmente em Haskell. Abaixo vamos definir uma função que realiza uma operação de divisão e, se ocorrer uma exceção devido a uma divisão por zero, tentar novamente por um número determinado de vezes. Abaixo vamos implementar a função RetentarDivisão, que realiza a divisão entre dois números, que implementa a capacidade de recuperação de falhas por retentativas:

```
import Control.Monad (when)

//Função de divisão com retentativas
RetentarDivisão:: Double -> Double -> Int -> IO (Either String Double)
RetentarDivisão _ _ 0 = return $ Left "Número máximo de tentativas
atingido."
RetentarDivisão x 0 _ = return $ Left "Divisão por zero não é
permitida."
RetentarDivisão x y attemptsLeft = do
  let result = x / y
  if isNaN result
    then do
      putStrLn "Ocorreu uma exceção: divisão por zero."
      putStrLn $ "Tentativas restantes: " ++ show (attemptsLeft - 1)
      RetentarDivisão x y (attemptsLeft - 1)
  else return $ Right result

main :: IO ()
main = do
  putStrLn "Digite o numerador:"
  inputNumerator <- getLine
  putStrLn "Digite o denominador:"
  inputDenominator <- getLine

  let numerator = read inputNumerator :: Double
    denominator = read inputDenominator :: Double
    maxAttempts = 3

  result <- RetentarDivisão numerator denominator maxAttempts

  case result of
    Left errorMsg -> putStrLn $ "Erro: " ++ errorMsg
    Right res -> putStrLn $ "Resultado da divisão: " ++ show res
```

Neste exemplo, definimos a função RetentarDivisão que recebe o numerador, denominador e o número máximo de retentativas permitidas. Se o denominador for zero, a função retornará Left com uma mensagem de erro. Caso contrário, tentará fazer a divisão e verificar se o resultado é um valor inválido (NaN, que ocorre quando dividimos por zero).

Se o resultado for inválido, mostramos uma mensagem de exceção, informando as tentativas restantes e chamamos a função RetentarDivisão novamente, reduzindo o número de tentativas em 1. O processo continuará até que o resultado seja válido ou até que o número máximo de tentativas seja atingido. Ao usar RetentarDivisão no main, agora podemos fazer divisões com retentativas, tentando novamente por um número limitado de vezes caso ocorra uma exceção de divisão por zero.

Outro mecanismo de recuperação de falhas bem popular é o uso de **supervisores.** Em sistemas concorrentes ou distribuídos, adota-se o modelo de **agentes supervisores**,

que monitoram e gerenciam outros componentes do sistema. Quando um componente falha, o supervisor pode reiniciá-lo ou tomar outras medidas de recuperação adequadas, garantindo assim a continuidade da execução.

A implementação de supervisores para recuperação de falhas, em Haskell, é comumente feita usando o conceito de processos leves (lightweight threads) chamados MVars. Os MVars permitem coordenar e sincronizar ações entre threads e podem ser usados para reiniciar ou lidar com falhas em componentes de forma supervisionada.

Abaixo apresentaremos o código de um supervisor que monitora um componente que simula um serviço de contador. Se o contador atingir um valor negativo, consideraremos isso uma falha, e o supervisor reiniciará o componente com um valor inicial padrão.

```
import Control.Concurrent (forkIO, threadDelay)
import Control.Concurrent.MVar (MVar, newMVar, takeMVar, putMVar)
type Counter = MVar Int

//Função que simula o componente do contador
agenteContador :: Counter -> IO ()
agenteContador auxiliar = do

//Simula um loop incrementando o contador
  loop
  where
    loop = do
      valorAtual <- takeMVar auxiliar
      let novoValor = valorAtual + 1
      putMVar auxiliar novoValor
      putStrLn $ "Contador: " ++ show novoValor
//Simula uma possível falha quando o contador atinge valor negativo
      if novoValor >= 0
        then loop
        else error "Falha no contador!"

//Função que cria um novo componente do contador supervisionado
supervisedCounter :: Int -> IO Counter
supervisedCounter valorInicial = do
  auxiliar <- newMVar valorInicial
  forkIO (agenteContador auxiliar)
  return auxiliar

main :: IO ()
main = do
  putStrLn "Iniciando o supervisor do contador..."
  auxiliar <- supervisedCounter 1
//Mantém o programa em execução para observar o contador
  threadDelay (2 * 1000000)
  putStrLn "Encerrando o programa."
```

Voltar ao sumário

Neste exemplo, usamos a biblioteca Control.Concurrent para criar um supervisor que monitora o componente agenteContador. O supervisor cria um MVar chamado auxiliar, que representa o estado do contador e é compartilhado entre o supervisor e o componente.

O componente agenteContador é simulado por um loop que incrementa o contador e imprime o seu valor. No entanto, quando o valor do contador se torna negativo (simulando uma falha), o componente lança uma exceção usando a função error.

O supervisor, implementado em supervisedCounter, cria um novo thread usando forkIO, que é responsável por executar o componente agenteContador. Se ocorrer uma falha no componente, o supervisor poderá detectá-la e reiniciar o componente com um valor inicial. O programa principal mantém o programa em execução por um período de tempo para que possamos observar o comportamento do contador e do supervisor.

Em sistemas que processam grandes volumes de dados ou que executam tarefas complexas, o **checkpointing** permite que o estado do sistema seja salvo em intervalos regulares. Isso possibilita reiniciar o sistema a partir de um ponto anterior em caso de falha, reduzindo a perda de progresso e de dados.

O uso de **backup e redundância** em aplicações críticas é comum para garantir que, caso um componente falhe, outro possa assumir suas funções. Além disso, a realização de backups periódicos dos dados garante que informações valiosas não sejam perdidas em caso de falha.

Transações ACID em sistemas envolvem bancos de dados, transações ACID (Atomicidade, Consistência, Isolamento e Durabilidade), que são fundamentais para garantir que as operações sejam executadas de forma segura e consistente, mesmo diante de falhas. A implementação de **monitoramento e registro de logs** em um sistema de monitoramento contínuo e registro detalhado de logs pode ajudar a identificar falhas e suas causas rapidamente, permitindo uma resposta mais ágil e eficiente.

Quando se trata de padrão de projetos, o **circuit breaker** é usado para evitar a propagação de falhas em cascata. Quando um serviço ou componente falha repetidamente, o circuit breaker interrompe as solicitações para esse componente, evitando sobrecarregar ainda mais o sistema e permitindo que ele se recupere antes de tentar novamente.

Neste Circuito, abordamos o conceito de recuperação de falhas e seu papel fundamental no desenvolvimento de aplicações robustas e confiáveis. Exploramos mecanismos de recuperação, como tratamento de exceções, retentativas e o uso de supervisores para lidar com falhas de forma mais controlada e eficiente.

A importância da recuperação de falhas não pode ser subestimada, especialmente em um mundo cada vez mais dependente de sistemas computacionais. A capacidade de uma aplicação se recuperar de erros e falhas é essencial para garantir a disponibilidade contínua do serviço, melhorar a experiência do usuário, reduzir prejuízos financeiros e manter a confiança dos clientes. Sistemas que lidam com dados críticos ou operações que podem ter consequências graves, como na área médica, financeira ou industrial, exigem uma atenção ainda maior à recuperação de falhas. A eficácia desses mecanismos pode determinar a segurança e o sucesso de um projeto.

Chegamos ao final do nosso quarto e último Percurso de Aprendizagem. É nosso desejo que você tenha assimilado todo o conteúdo aqui apresentado. Que tal agora você responder aos exercícios de fixação e conferir o seu aprendizado?

Voltar ao sumário

REFERÊNCIAS

ASCENCIO, Ana Fernanda Gomes; CAMPOS, Edilene Aparecida Veneruchi de. **Fundamentos da programação de computadores**: algoritmos, pascal, c/c++ e java. 3. ed. São Paulo: Pearson Education do Brasil, 2012. (Cód.:83968)

GHEZZI, Carlo. **Conceitos de linguagens de programação**. Colaboração de Mehdi Jazayeri. Tradução Paulo A. S Veloso. 2. ed. Rio de Janeiro: Campus, 1987. (Cód.:5506)

SEBESTA, Robert W. **Conceitos de linguagens de programação**. 11. ed. Porto Alegre: Bookman, 2018. Disponível em: https://integrada.minhabiblioteca.com.br/books/9788582604694. (DIGITAL) (Cód.:23617)

UNIVERSIDADE DE FORTALEZA (UNIFOR)

Presidência
Lenise Queiroz Rocha

Vice-Presidência
Manoela Queiroz Bacelar

Reitoria
Randal Martins Pompeu

Vice-Reitoria de Ensino de Graduação e Pós-Graduação
Maria Clara Cavalcante Bugarim

Vice-Reitoria de Pesquisa
José Milton de Sousa Filho

Vice-Reitoria de Extensão
Thiago Braga Martins

Vice-Reitoria de Administração
José Maria Gondim Felismino Júnior

Diretoria de Comunicação e Marketing
Ana Leopoldina M Quezado Vargas Valle

Diretoria de Planejamento
Marcelo Nogueira Magalhães

Diretoria de Tecnologia
José Eurico de Vasconcelos Filho

Diretoria do Centro de Ciências da Comunicação e Gestão
Danielle Batista Coimbra

Diretoria do Centro de Ciências da Saúde
Lia Maria Brasil de Souza Barroso

Diretoria do Centro de Ciências Jurídicas
Katherinne de Macêdo Maciel Mihaliuc

Diretoria do Centro de Ciências Tecnológicas
Jackson Sávio de Vasconcelos Silva

AUTOR
LEANDRO DA SILVA TADDEO

Bacharel em Informática, Mestre em Informática Aplicada, 26 anos de experiência em docência, 30 anos de experiência em TI. Pesquisador, Professor de graduação em Sistemas de Informação e professor de Pós-graduação em Design Digital. Atualmente, atua como analista judiciário, lotado no setor de informática do Fórum Clovis Beviláqua.

RESPONSABILIDADE TÉCNICA

Universidade de Fortaleza | VRE Vice-Reitoria de Ensino de Graduação e Pós-Graduação

Universidade de Fortaleza | EAD Educação a Distância

COORDENAÇÃO GERAL DE EDUCAÇÃO A DISTÂNCIA

Coordenação Geral
Andrea Chagas Alves de Almeida

Supervisão de Equipe Multidisciplinar
Francisco Weslley Lima

Analista de Mídias
Emanoel Alves Cavalcante

Design Instrucional
Jéssica Gabrielle de Menezes Lima

Revisão Gramatical
José Ferreira Silva Bastos

Identidade Visual / Arte
Thiago Bruno Costa de Oliveira

Editoração / Diagramação
Rebeka Melo Peres

Produção de Áudio e Vídeo
José Moreira de Sousa

Programação / Implementação
Renan Alves Diniz
Thais Rozas Teixeira